

学 号：	2025302919
------	------------

研究生课程报告

武汉理工大学

课程名称	企业级应用软件设计与开发
项目名称	KnowledgeOps Agent 企业知识库自治运营智能体
方向	Agentic AI 原生开发
学号	2025303031
姓 名	陈政宇
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 21 日

目录

..... 1

研究生课程报告 1

一、选题背景与设计思想 4

 1.1 问题定义 4

 1.2 设计思想 4

 1.3 项目价值 5

 1.4 技术路线 6

二、Specs 规格文档 6

 2.1 Product Spec 6

 2.2 Architecture Spec 7

 2.3 API Spec 8

 2.4 Data Spec 8

三、系统架构与设计 9

 3.1 总体架构 9

 3.2 知识库与文档管理功能 10

 3.3 意图管理与问题路由功能 11

 3.4 RAG 问答与链路追踪功能 12

 3.5 Agent 自治运营功能 14

四、关键实现与代码展示 15

 4.1 Agent 核心循环 15

 4.2 工具抽象与注册机制 15

 4.3 BenchmarkTool 与知识缺口分析 16

 4.4 兜底回答配置 16

五、测试与评估 17

 5.1 测试目标与整体思路 17

 5.2 功能测试 17

 5.3 RAG 单条问题测评 18

 5.4 Benchmark 批量评测流程 20

 5.5 当前测试结论 21

- 六、系统升级与扩展 21
 - 6.1 可扩展架构 21
 - 6.2 下一阶段计划 22
 - 6.3 AI 能力演进路径 22
- 七、课程总结 23
 - 7.1 个人收获 23
 - 7.2 工程思维转变 23
 - 7.3 对课程的建议 24

一、选题背景与设计思想

1.1 问题定义

本项目围绕企业知识库的“可用、可信、可运营”问题展开。企业知识库在制度、产品、研发、运维和客户支持等场景中持续积累，但传统系统通常只解决文档存储和关键词检索问题。随着文档来源增加、格式复杂、业务变化加快，知识库是否覆盖关键问题、检索证据是否可靠、分块质量是否健康，逐渐成为比“能否上传文档”更重要的问题。

基础 RAG 系统可以根据文档回答问题，但它往往只关注一次问答链路，很难主动回答“这批文档是否足够支撑某个业务场景”“哪些问题无法命中证据”“哪些文档或分块影响检索效果”。因此，本项目将目标定义为从零创建一个面向企业知识库自治运营的 Agentic RAG 系统，使知识库从被动资料库升级为可诊断、可评估、可持续优化的知识基础设施。

1.2 设计思想

项目的设计思想是将企业知识库从“文档存储系统”升级为“可持续运营的知识服务系统”。传统知识库通常只关注文档是否能够上传、是否能够被检索，而本项目更关注知识进入系统之后，是否能够被正确理解、有效检索、可靠回答，并在使用过程中持续发现问题、修正问题。因此，系统围绕“知识库建设、知识问答、知识运营”三个环节形成闭环，让知识库不再只是被动保存资料，而是能够被评估、被诊断、被优化的企业知识基础设施。

在知识库建设阶段，系统通过数据通道完成企业文档的接入与处理。管理员上传制度文件、产品文档、技术说明或业务手册后，系统不会直接把原始文件用于问答，而是先对文档进行解析、清洗、切分和增强。解析过程负责把 PDF、Word、Markdown 等不同格式的文件转化为统一文本；切分过程将长文档拆解为适合检索的知识片段；增强过程可以补充标题、来源、层级、关键词等元信息，使后续检索能够更准确地定位内容。最终，处理后的知识片段会写入关系数据库和向量索引，为 RAG 问答和 Agent 评估提供基础数据。

在知识问答阶段，系统通过 RAG 链路把用户问题转化为可检索、可回答的任务。用户提出问题后，系统首先结合上下文进行问题改写，解决多轮对话中指代不清、问题过短或表达不完整的问题。随后，意图识别模块判断该问题属于普通知识问答、业务流程咨询、系统操作问题，还是需要调用外部工具的任务。进入知识库问答流程后，系统会执行多通道检索，从关键词、向量语义、历史上下文等多个角度召回候选知识片段，再通过重排模型筛选出最相关的证据。最后，大模型基于检索到的企业知识片段生成回答，并尽量保留引用依据，使回答不是单纯依赖模型通用知识，而是建立在企业文档证据之上。

在知识运营阶段，系统引入 Agent 对知识库质量进行主动评估。管理员可以发起覆盖率评估、Benchmark 测试、知识缺口分析、文档新鲜度检查等运营任务。Agent 接收到任务目标后，会根据任务场景选择合适的工具链，例如使用 BenchmarkTool 对一组标准问题进行检索测试，使用知识缺口分析工具定位低命中问题，使用覆盖率评估工具判断知识库对业务场景的支撑程度。执行过程中，系统会记录每一步工具调用的输入、输出、状态和耗时，使管理员能够看到 Agent 如何一步步完成诊断，而不是只得到一个不可解释的最终结论。

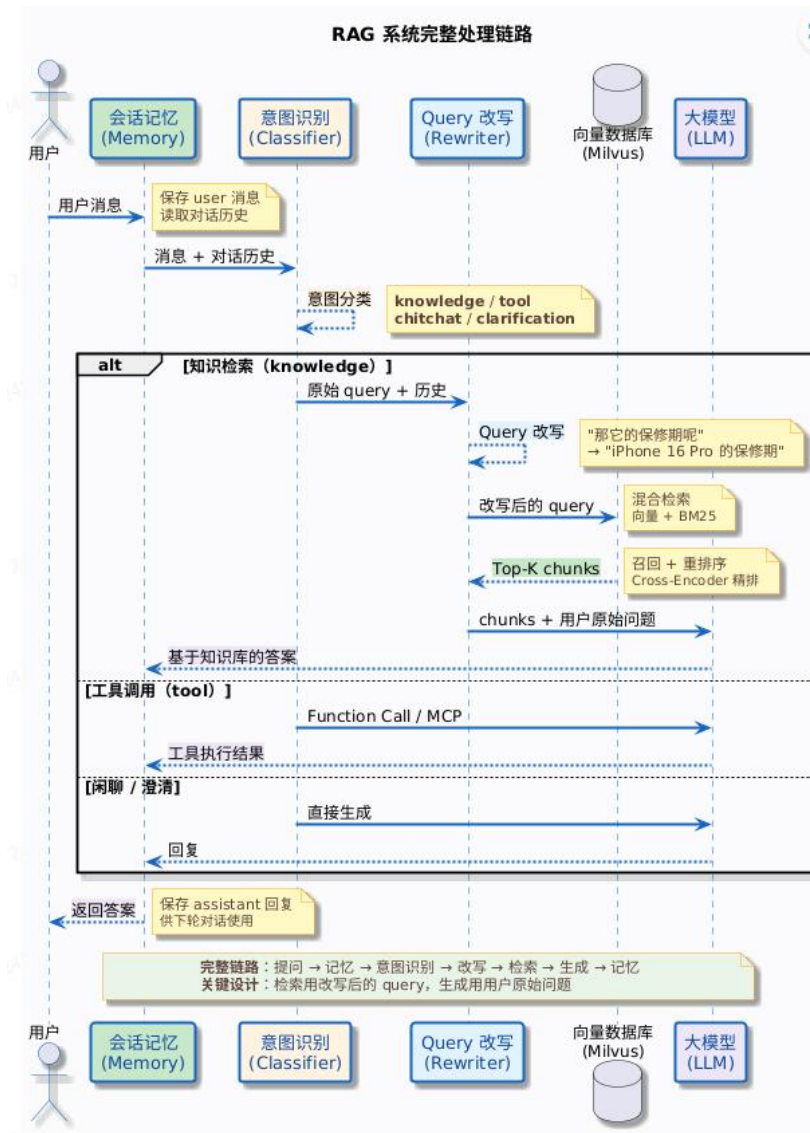


图 1：RAG 系统运行流程

1.3 项目价值

从用户角度看，系统不仅能在有知识库证据时生成带依据的回答，也能在检索为空或置信度不足时进入受控兜底回答，避免基础问题无法响应。兜底回答会明确提示缺少知识库证据，从而区分“企业文档依据”和“通用模型知识”。

从管理员角度看，系统将人工巡检知识库的过程工具化。BenchmarkTool 可以用问题集衡量知识库对目标任务的支撑能力，知识缺口分析可以从低命中问题中定位薄弱主题，覆盖率评估可以把工具结果转化为健康度和改进建议。知识库维护因此不再只依赖经验，而是有指标、有证据、有后续动作。

从工程角度看，项目采用前后端分离、多模块后端、配置化模型路由、运行记录持久化和 Trace 设计。功能展示截图主要覆盖用户能看到的系统能力，核心实现则通过代码片段解释关键逻辑，两者共同证明系统既能运行，也有清晰的工程结构。

1.4 技术路线

后端采用 Spring Boot、MyBatis-Plus、Sa-Token 和 Maven 多模块工程，前端采用 React、Vite、TypeScript、Tailwind CSS 和 Zustand。数据与基础设施包括 PostgreSQL/pgvector、Milvus、Redis、RocketMQ、RustFS/S3 和 Docker Compose。AI 能力层面，课程目标模型为 DeepSeek API，工程结构也保留 Bailian、SiliconFlow、AIHubMix、Ollama 等模型提供方扩展能力。

RAG 能力包括 Query Rewrite、多通道检索、Rerank、Prompt 组装、会话记忆、SSE 流式输出和 fallback 兜底回答。Agent 能力采用 Planner、Executor、Tool Registry 和 Reporter 的自研实现，借鉴 LangGraph 的状态流转和工具节点思想，但结合企业知识库运营场景实现更受控的工具编排。

二、Specs 规格文档

2.1 Product Spec

2.1.1 产品定位

KnowledgeOps Agent：企业知识库自治运营智能体，定位为面向企业知识库场景的 Agentic AI 原生系统。它不是单纯的聊天机器人，也不是普通的向量检索应用，而是将文档入库、知识问答、知识库评估、知识缺口分析和改进建议生成连接在一起的完整系统。

系统的核心目标是解决企业知识库长期维护中的三个问题：第一，知识是否能够被有效组织并进入可检索状态；第二，用户提问时系统是否能够基于企业知识给出可信回答；第三，知识库质量是否能够被持续评估，并反向指导文档补充和检索策略优化。因此，项目不是只追求一次问答效果，而是强调知识库全生命周期的运营能力。

系统主要面向四类使用者。普通员工通过问答界面获取制度、流程、产品和技术知识；知识库管理员负责维护知识库、文档、意图和数据通道；研发与运维人员通过 Trace、Run、Step 等记录排查 Agent 和 RAG 链路问题；管理者通过运营报告了解知识库覆盖情况、薄弱主题和下一阶段建设重点。

2.1.2 功能规格

功能规格围绕两条主线展开。一条是面向用户的知识问答主线，要求系统能够管理知识库、处理文档、执行语义检索并生成回答；另一条是面向管理员的知识运营主线，要求系统能够记录 Agent 运行、执行工具链、评估检索质量并输出改进建议。

功能模块	规格说明	核心输出
Dashboard	展示系统运行概况，包括知识库数量、文档处理状态、模型状态、问答趋势和 Agent 任务情况。	系统总览指标
知识库管理	支持知识库创建、文档上传、文档状态查看、Chunk 管理和数据集维护。	可检索的知识资产
数据通道	负责文档解析、文本切分、内容增强、索引写入和处理任务追踪。	文档处理流水线记录
意图管理	维护业务意图、系统意图和工具意图，用于判断用户问题应该进入问答、检索还是工具调用流程。	意图路由规则
RAG 问答	支持多轮问答、问题改写、语义检索、重排、引用证据和兜底回答。	用户答案与证据片段
链路追踪	记录一次问答中的检索结果、Prompt、模型响应、耗时和错误信息。	Trace 诊断记录
Agent 运营	支持创建知识库运营任务，查看 Run、Step、工具输出和最终报告。	Agent 执行过程与报告
Benchmark 评估	使用问题集测试知识库覆盖能力，计算命中率、Top Score、未命中问题等指标。	评估指标与问题明细
知识缺口分析	基于低命中问题识别薄弱主题，生成文档补充和知识库优化建议。	缺口列表与改进建议

2.1.3 非功能规格

系统具备可扩展性。当新 Agent 工具时，可以通过实现 AgentTool 接口并注册为 Spring Bean 接入，而不是改动核心执行器。系统还需要具备可维护性，模型调用、检索、知识库管理、Agent 编排和前端展示应保持清晰边界，避免业务逻辑直接耦合具体模型供应商。

系统还要具备可观测性和容错性。每一次问答应记录 Trace，每一次 Agent 运行都记录 Run 和 Step，每一步工具调用的输入输出都可以查询。检索为空或低置信时，系统不应该简单拒答，而应进入受控兜底逻辑。模型调用失败时，可以通过候选模型和路由策略降来低单点失败影响。

2.2 Architecture Spec

系统采用前后端分离与后端多模块架构。frontend 承载用户界面，bootstrap 是主业务应用，infra-ai 负责模型调用抽象，framework 提供通用工程能力，mcp-server 提供外部工具服务示例，resources 保存数据库脚本、Docker Compose 和示例知识库。

src/	
└ bootstrap/	主应用: RAG、Agent、知识库、文档摄取、管理端接口
└ infra-ai/	LLM、Embedding、Rerank、模型路由、健康检查
└ framework/	通用工程能力: 异常、上下文、Trace、Redis、MQ
└ mcp-server/	MCP 工具服务端示例
└ frontend/	React 前端: 聊天、知识库、Trace、Agent 运营
└ resources/	数据库脚本、Docker Compose、示例知识库

2.3 API Spec

Agent 自治运营模块以 Run 作为核心资源。一次 Run 代表一次完整的知识库运营任务，例如对某个知识库执行覆盖率评估、Benchmark 测试或知识缺口分析。管理员在前端提交任务目标后，后端会创建对应的 Run 记录，并根据任务场景触发 Planner 生成执行计划。随后 Executor 按计划调用 BenchmarkTool、知识缺口分析工具、覆盖率评估工具等能力，系统会把每一步工具调用记录为 Step，用于后续追踪和复盘。

前端围绕 Run 和 Step 展示 Agent 的执行过程。Run 用于表示整体任务，包括任务目标、执行状态、开始时间、结束时间和最终报告。Step 用于表示任务中的单个执行步骤，包括工具名称、输入参数、输出结果、执行状态和耗时。这样设计的好处是，用户不仅能看到最终结论，也能看到 Agent 是如何一步步得到结论的，从而提升系统的可解释性和可审计性。

API	方法	说明
/agent/knowledge-ops/runs	POST	启动一次知识库自治运营任务。
/agent/knowledge-ops/runs	GET	分页查询 Agent 运行记录。
/agent/knowledge-ops/runs/{run-id}	GET	查询单次运行详情和结构化报告。
/agent/knowledge-ops/runs/{run-id}/steps	GET	查询单次运行的步骤列表，支持工具级复盘。

```
POST /agent/knowledge-ops/runs
{
  "kbId": "kb_001",
  "task": "评估该知识库是否能够支撑新人入职制度问答",
  "scenario": "benchmark",
  "topK": 8,
  "benchmarkQuestions": ["新人试用期多久?", "入职材料需要哪些?"]
}
```

2.4 Data Spec

数据规格围绕运行记录、步骤记录、知识片段和问答 Trace 展开。Agent Run 表示一次完整的自治运营任务，Agent Step 表示任务中的单个工具执行步骤，Knowledge Chunk 是 RAG 检索的基本单位，Trace 保存问答链路中的检索、Prompt 和生成信息。

数据对象	关键字段	作用
Agent Run	id、userId、kbld、task、status、summary、reportJson、startedAt、finishedAt	描述一次知识库自治运营任务的生命周期。
Agent Step	runId、stepOrder、stepType、toolName、status、inputJson、outputJson、errorMessage	描述工具链中每一步的执行过程和结果。
Knowledge Chunk	id、kbld、docId、content、contentHash、enabled、deleted	作为语义检索和证据引用的基本知识单元。
Trace	conversationId、request、retrieval、prompt、response、error	支撑问答链路回放、问题定位和效果分析。

三、系统架构与设计

3.1 总体架构

系统整体由前端交互层、应用服务层、RAG 能力层、Agent 工具层、AI 基础设施层和数据基础设施层组成。前端负责把知识库管理、问答、Trace 和 Agent 运营过程呈现给用户；后端负责文档入库、检索、生成、工具编排和运行记录；外部基础设施提供向量检索、缓存、消息队列、对象存储和模型服务。

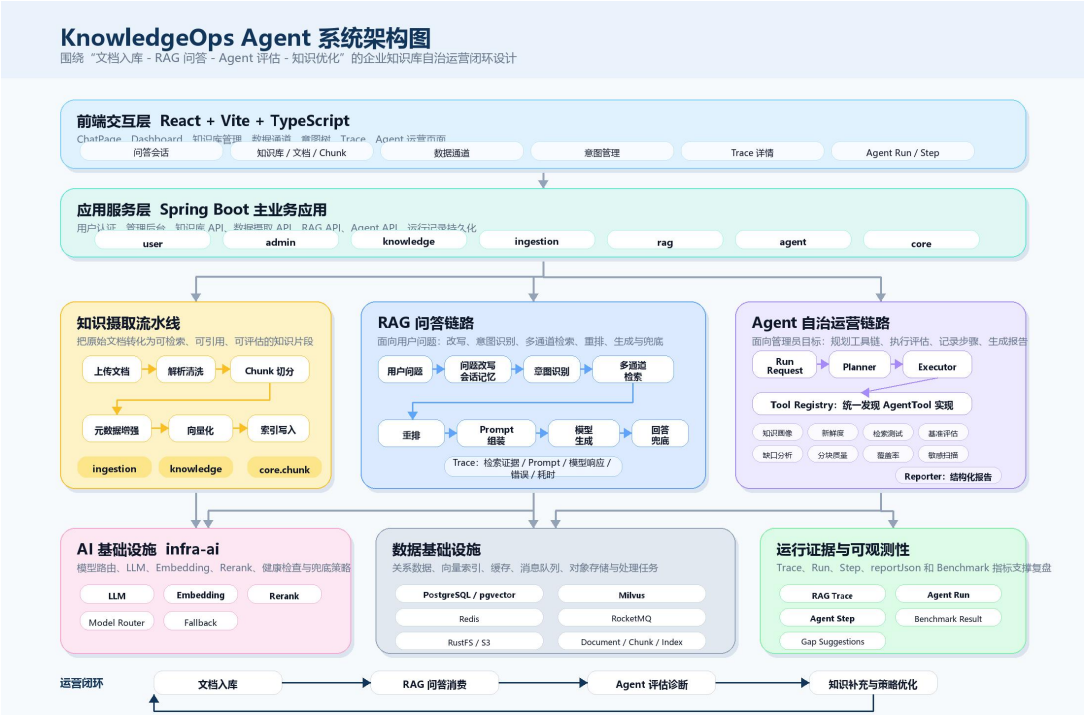


图 2：系统模块分层架构

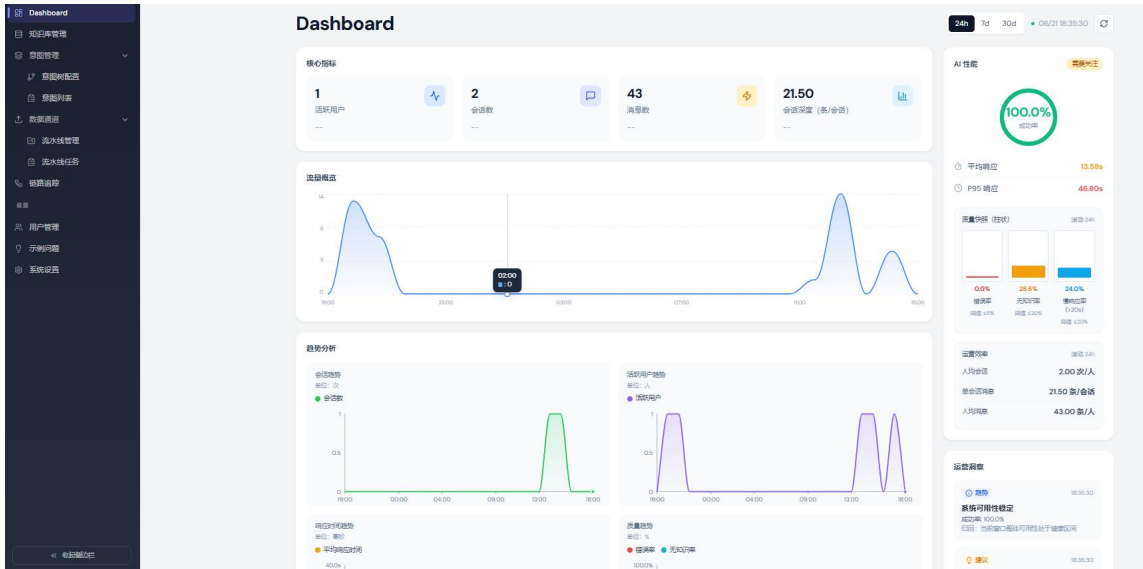


图 3: Dashboard 前端页面展示

3.2 知识库与文档管理功能

知识库管理是系统的数据入口。管理员可以在管理端创建知识库、上传文档、查看文档解析状态和 Chunk 信息。该功能对应后端的知识库、文档和分块管理模块，是后续 RAG 检索和 Agent 评估的基础。



图 4: 知识库文档管理页面展示

文档进入系统后并不会直接用于回答，而是先经过 pipeline 流水线。流水线把原始文件解析为文本，再切分为可检索 Chunk，随后写入关系数据库和向量数据库。这个过程决定了问答效果的上限，也是 Agent 后续能够检查新鲜度、Chunk 质量和覆盖率的前提。



图 5:文档提取流程展示



图 6:数据通道前端页面展示

3.3 意图管理与问题路由功能

意图管理用于维护系统能够识别的业务意图、系统意图和 MCP 工具意图。它决定了用户问题进入系统后是直接回答、检索知识库，还是调用外部工具。你提供的意图列表截图展示了意图名称、层级、类型、路径、关联资源、示例数和启用状态，这部分可以作为系统功能完整性的直接证据。

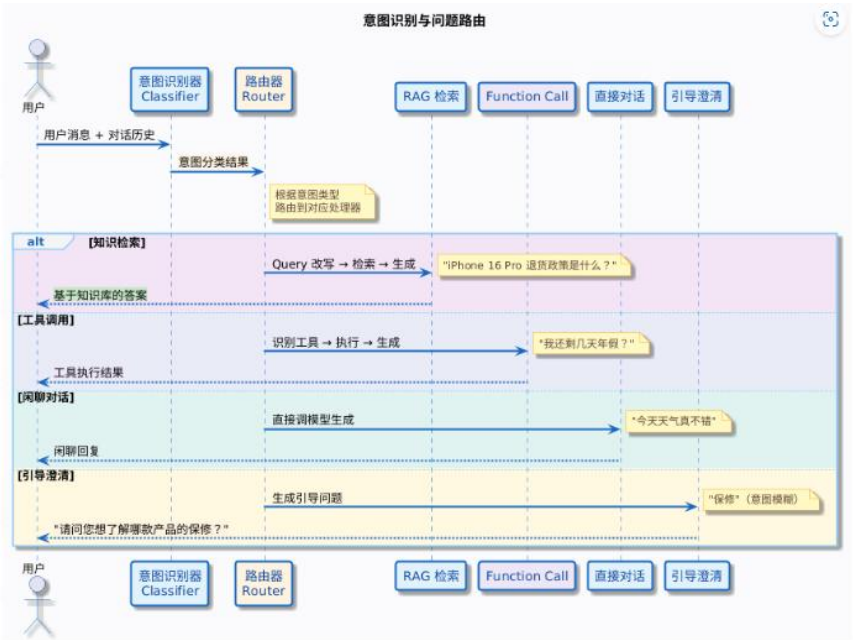


图 7:意图识别与问题展示图

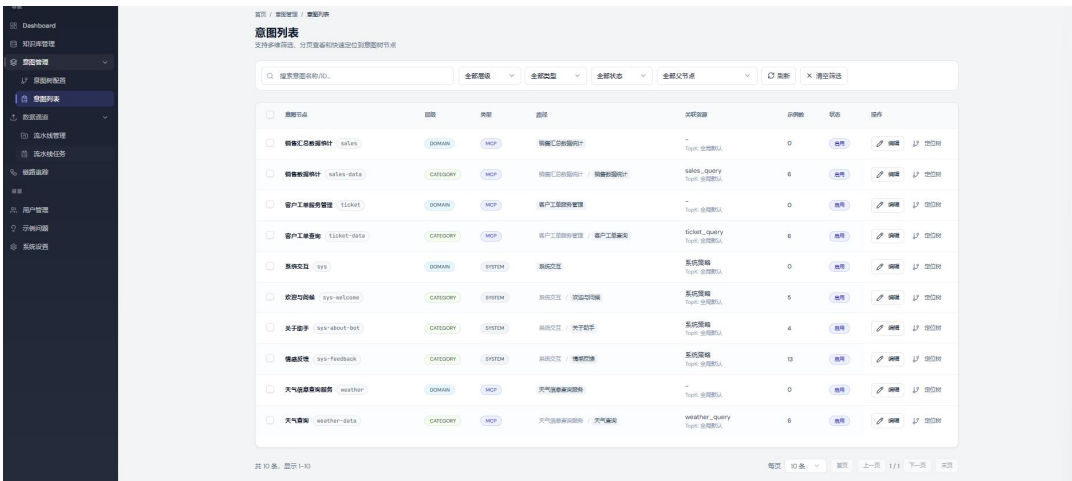


图 8:意图列表功能前端页面

3.4 RAG 问答与链路追踪功能

RAG 问答链路以用户问题为入口，经过会话记忆、问题改写、意图识别、多通道检索、Rerank、Prompt 组装和模型生成。系统不仅输出答案，也会保留检索证据和Trace 信息，便于管理员理解回答为什么成功或失败。

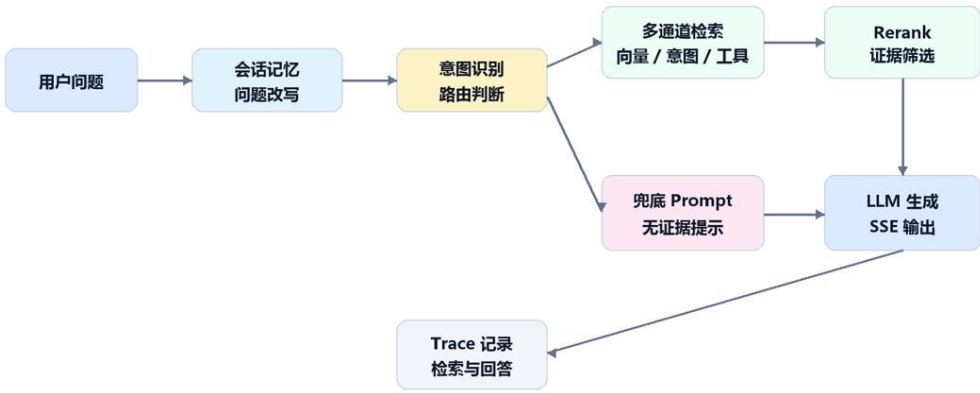


图 9:RAG 问答与兜底流程展示



图 10:多通道检索设计



图 11:系统问答展示图

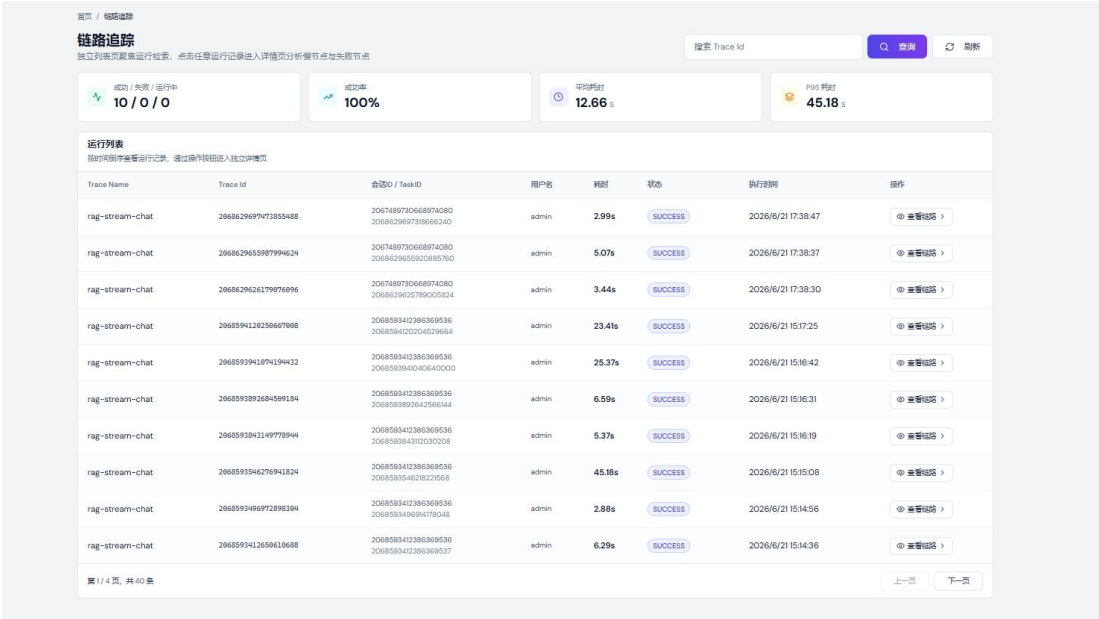


图 12: 链路追踪 Trace 详情页面

3.5 Agent 自治运营功能

Agent 运营功能用于从管理员视角评估知识库质量。管理员输入运营目标后，Planner 选择 workflow，Executor 顺序调用工具，Step Recorder 记录每一步输入输出，Reporter 汇总形成报告。该流程让知识库运营从人工经验判断变为可执行、可量化、可复盘的任务。

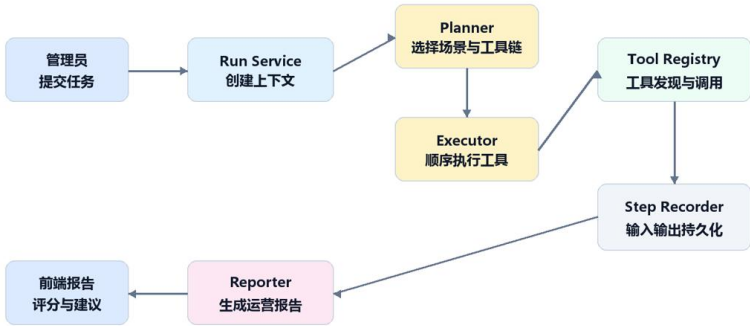


图 13: Knowledges Agent 运行架构



图 14: idea 中运行展示

四、关键实现与代码展示

4.1 Agent 核心循环

Agent 核心循环位于 KnowledgeOpsAgentServiceImpl.run(), 它负责创建 Run、构造上下文、调用 Planner、执行工具链、生成报告并更新运行状态。

```
KnowledgeOpsRunDO run = KnowledgeOpsRunDO.builder()
    .userId(UserContext.getUserId())
    .kbId(request.getKbId())
    .task(request.getTask())
    .status(KnowledgeOpsRunStatus.RUNNING.name())
    .startedAt(new Date())
    .build();
runMapper.insert(run);

AgentPlan plan = planner.plan(context);
context.setPlan(plan);
executor.execute(plan, context);
KnowledgeOpsReport report = reporter.buildReport(context);
run.setStatus(KnowledgeOpsRunStatus.SUCCESS.name());
run.setReportJson(JSONUtil.toJsonStr(report));
runMapper.updateById(run);
```

这段代码体现了项目的核心控制流：Run 记录保证任务可追踪，Context 负责在工具之间传递状态，Planner 和 Executor 分离了规划与执行，Reporter 将工具结果转化为用户可读报告。

4.2 工具抽象与注册机制

AgentTool 是工具扩展的统一接口。每个工具都声明自己的名称、类型、描述、输入输出 Schema 和 execute 方法。这样新增工具时只需实现接口并注册为 Spring Bean，Executor 不需要知道工具内部细节。


```
public interface AgentTool {
    String name();
    String type();
    default Map<String, String> inputSchema() { ... }
    default Map<String, String> outputSchema() { ... }
    default boolean supportRetry() { return false; }
    default int timeoutSeconds() { return 30; }
    AgentToolResult execute(KnowledgeOpsContext context);
}
```

工具注册表通过 Spring 容器收集所有 AgentTool 实现，并按工具名建立映射。Executor 执行 AgentPlan 时，根据 step 中的 toolName 查找工具并执行，这使工具链可以由配置文件控制，也便于后续扩展新的评估能力。

4.3 BenchmarkTool 与知识缺口分析

BenchmarkTool 是知识库自治运营的关键实现。它会从请求参数、任务描述和样例问题库中得到评测问题集，对每个问题执行检索，然后根据 topScore 是否超过阈值判断是否命中。最终输出 questionCount、hitRate、averageTopScore、missCount 和 items 等结构化指标。

```
for (String question : questions) {
    List<RetrievedChunk> chunks = retrieverService.retrieve(
        RetrieveRequest.builder()
            .query(question)
            .topK(topK)
            .collectionName(kb.getCollectionName())
            .build()
    );
    RetrievedChunk top = chunks.isEmpty() ? null : chunks.get(0);
    float topScore = top == null ? 0F : top.getScore();
    item.put("hit", topScore >= minScore);
    item.put("topScore", topScore);
    item.put("evidenceCount", chunks.size());
}
```

知识缺口分析工具读取 BenchmarkTool 输出，将未命中或低分问题识别为 weakQuestions，并结合证据数量生成 gapLevel 和 suggestions。它把检索评测从数字指标进一步转化为管理员可以执行的知识补充建议。

4.4 兜底回答配置

针对没有相关文档时无法回答基础问题的问题，系统增加了 fallback 配置。开启后，检索为空或低置信时可以进入受控兜底回答，但回答需要说明没有命中企业知识库证据。

```
rag:
  fallback:
    enabled: true
```

```
direct-basic-question-enabled: true
low-confidence-enabled: true
min-relevant-score: 0.35
```

```
c.n.a.r.r.c.i.DefaultIntentClassifier : 当前问题: 今天武汉的天气

.n.a.r.r.c.r.c.VectorGlobalSearchChannel : 未识别出任何意图, 启用全局检索
.n.a.r.r.c.r.c.MultiChannelRetrievalEngine : 启用的检索通道: [VectorGlobalSearch]
.n.a.r.r.c.r.c.MultiChannelRetrievalEngine : 执行检索通道: VectorGlobalSearch
.n.a.r.r.c.r.c.VectorGlobalSearchChannel : 执行向量全局检索, 问题: 今天武汉的天气
.n.a.r.r.c.r.c.AbstractParallelRetriever : 全局检索 检索统计 - 总目标数: 1, 成功: 1, 失败: 0, 检索到 Chunk 总数: 4
.n.a.r.r.c.r.c.VectorGlobalSearchChannel : 向量全局检索完成, 检索到 4 个 Chunk, 耗时 1872ms
.n.a.r.r.c.r.c.MultiChannelRetrievalEngine : 通道 VectorGlobalSearch 完成 ✓ - 检索到 4 个 Chunk, 耗时: 1872ms
.n.a.r.r.c.r.c.MultiChannelRetrievalEngine : 多通道检索统计 - 总通道数: 1, 有结果: 1, 无结果: 0, Chunk 总数: 4
.n.a.r.r.c.r.c.MultiChannelRetrievalEngine : 后置处理器 Deduplication 完成 - 输入: 4 个 Chunk, 输出: 4 个 Chunk, 变化: 0
.n.a.r.r.c.r.c.MultiChannelRetrievalEngine : 后置处理器 Rerank 完成 - 输入: 4 个 Chunk, 输出: 4 个 Chunk, 变化: 0
.n.a.r.r.c.r.c.MultiChannelRetrievalEngine : 后置处理器链执行完成 - 初始: 4 个 Chunk, 最终: 4 个 Chunk
.n.a.r.r.c.r.c.QueryTermMappingCacheManager : 术语映射已保存到 Redis 缓存, 共 0 条规则
c.n.a.r.r.c.r.QueryTermMappingService : 术语映射规则从数据库加载完成, 共 0 条规则
.n.a.r.r.c.r.c.MultiQuestionRewriteService : RAG用户问题查询改写+拆分:
```

图 15: 触发兜底机制时系统处理

五、测试与评估

5.1 测试目标与整体思路

本项目的测试不是单纯验证接口是否能够返回数据, 而是围绕“企业知识库是否能够被构建、被问答、被追踪、被评估、被改进”这一完整闭环展开。

KnowledgeOps Agent 的核心能力包括文档摄取、知识库管理、RAG 问答、链路追踪、Agent 运营任务、Benchmark 评测和知识缺口分析。因此, 测试也按照这条业务路径组织, 先验证系统功能是否能够贯通, 再验证问答与评估结果是否具有可复盘价值。

测试流程可以分为两类。第一类是功能测试, 重点验证系统页面和核心业务路径是否可用, 例如登录、知识库管理、数据通道、问答页面、Trace 页面和 Agent Run 页面。第二类是评估测试, 重点验证 RAG 问答质量和 Agent 运营结果是否可信, 例如问题是否命中文档证据、检索到的 Chunk 是否相关、首 token 响应时间是否可接受、Benchmark 是否能够输出命中率和缺口建议。

结合本项目的评测流程图, 测试采用“单条问题评测 + 批量 Benchmark 评测”的方式。单条问题评测用于检查一次问答链路是否完整, 批量 Benchmark 评测用于检查知识库在一组标准问题上的整体表现。这样既能发现单次问答中的具体问题, 也能从统计结果上判断知识库覆盖能力。

5.2 功能测试

功能测试按照用户真实使用路径展开。首先启动后端、前端以及 PostgreSQL、Redis、Milvus、RocketMQ、对象存储等依赖服务, 确认系统能够正常访问。随后进入管

理端，检查知识库、文档、Chunk、数据通道和意图管理页面是否能够正常展示。最后通过问答页面发起 RAG 问答，并在 Trace 和 Agent 运营页面查看系统执行过程。

测试项	测试方法	预期结果
系统启动与登录	启动后端、前端和基础依赖，访问登录页面并进入系统。	前端可访问，登录后能够进入 Dashboard 或管理页面。
知识库管理	查看知识库列表、文档列表、Chunk 明细和文档处理状态。	文档状态清晰，Chunk 可查看，知识库数据能够被后续检索使用。
数据通道	上传或批量导入文档，触发文档解析、切分、增强和索引写入。	数据通道任务能够生成节点状态，文档最终进入可检索状态。
意图管理	查看意图列表、意图类型、层级结构、关联资源和启用状态。	意图数据能够参与问题路由，支撑业务意图识别和工具调用。
RAG 问答	针对已入库知识提出业务问题，观察回答和引用证据。	系统能够返回基于知识库的回答，并展示相关证据片段。
链路追踪	对一次问答查看 Trace 详情，包括检索、Prompt、模型响应和耗时。	Trace 能够回放问答链路，便于定位检索失败或回答异常。
兜底回答	在无相关文档时询问基础通用问题。	系统能够给出受控兜底回答，并提示未命中知识库证据。
Agent 运营	创建 benchmark、quality 或 retrieval 场景任务。	系统生成 Run、Step、工具输出和结构化报告。
Benchmark 与缺口分析	使用问题集评估知识库覆盖能力。	输出命中率、Top Score、低命中问题和知识补充建议。

功能测试的重点在于验证系统链路是否闭合。文档不能只停留在上传状态，而要能够被切分和索引；问答不能只返回自然语言，而要能够关联检索证据；Agent 任务不能只生成一个结论，而要能够记录 Run 和 Step，展示每个工具执行过程。只有这些环节都能够串联起来，才能说明系统具备完整的企业知识库自治运营能力。

5.3 RAG 单条问题测评

单条问题评测用于观察一次真实 Query 从客户端发起到系统返回回答的全过程。评测 Runner 作为外部 Python 客户端，先调用 /rag/v3/chat 接口发起 SSE 流式问答请

求。系统返回的 SSE 事件中包含 meta、message、finish 和 done 等信息。Runner 会记录 conversationId、taskId、思考过程、回答片段、首 token 时间和完整响应时间。这样可以同时评估回答内容和响应性能。

在第一阶段，Runner 关注用户真实问答体验。它会记录模型是否能够正常开始响应、是否出现思考片段、最终回答是否完整、首 token 延迟是否过高。对于 RAG 系统来说，首 token 时间和整体耗时非常重要，因为企业用户不仅要求回答准确，也要求交互体验稳定。如果检索、重排或模型调用过慢，即使最终答案正确，也会影响系统可用性。

第二阶段，Runner 会调用 /rag/eval 接口获取评测元数据。该接口返回的不只是答案文本，而是与评测相关的结构化信息，例如命中的文档 ID、Chunk ID、上下文片段、意图节点、是否命中知识库、是否调用 MCP 工具等。通过这些信息，可以判断回答到底是否来自知识库证据，而不是只依赖模型自身知识。

第三阶段，Runner 将 SSE 流式回答、Eval 元数据和评估集中的标准字段合并成一条 EvalRecord，并按时间戳写入 JSONL 文件。这样每一条问题都会形成可复盘记录，后续可以用于统计命中率、召回效果、回答质量和失败原因。这个设计使评测过程具备可追踪性，不再只是人工观察“回答看起来对不对”。

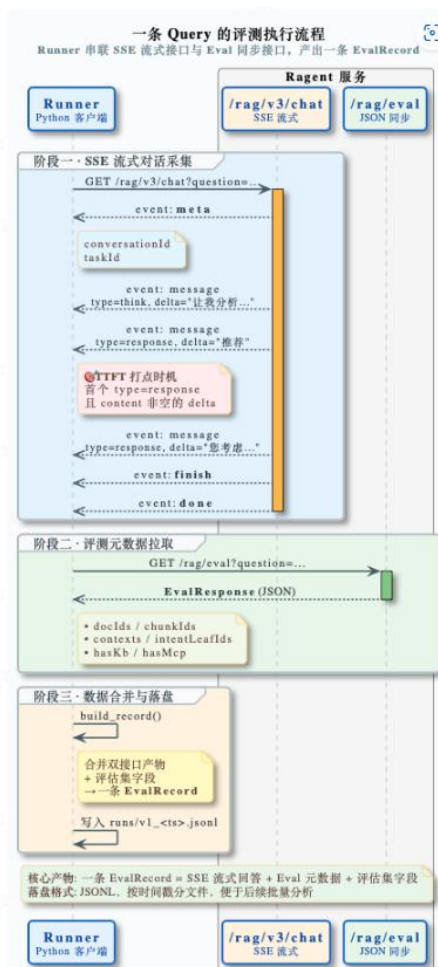


图 16：单条 query 评测执行流程

5.4 Benchmark 批量评测流程

Benchmark 批量评测用于将知识库评测从人工试问升级为可复现的自动化评估。整个流程可以分为 init、run、score 和 report 四个阶段，每个阶段都产生结构化产物，便于后续复盘和对比。

init 阶段负责准备评测环境。系统会预先准备 Markdown 业务文档，创建多个知识库，例如商品、手册、政策、FAQ 等不同类型的知识库，然后批量导入文档并触发分块处理。同时，系统会构建意图树，使后续问题能够经过意图识别和路由。初始化阶段会产出 kb_ids.json、doc_id_map.json 和 intent_ids.json 等文件，用于保证后续评测能够定位同一批知识库、文档和意图资源。

run 阶段负责执行问题集。Runner 读取 eval_set_v1.json 中的问题，对每个问题调用 /rag/v3/chat 获取 SSE 流式回答，再调用 /rag/eval 获取检索证据和评测元数据。系统会把每条问题的回答、证据、文档命中情况、Chunk 命中情况、意图命中情况和耗时数据合并保存，形成 runs/v1_<timestamp>.jsonl。该阶段的关键是保证每道题都有完整过程记录，而不是只保存最终答案。

score 阶段负责计算指标。评测指标分为两类：一类是自建指标，例如覆盖率、Hit@K、Recall、MRR、首 token 时间和整体延迟；另一类是 RAGAS 类指标，例如 faithfulness、relevancy、correctness、precision 和 recall。对于需要语义判断的内容，还可以引入 LLM-as-judge，对回答是否忠实于证据、是否回答了问题进行辅助评价。

report 阶段负责生成评测结果。系统可以输出 Markdown 报告、CSV 表格、失败样例 JSON 和 HTML 展示页。Markdown 报告适合直接放入课程报告，CSV 适合做指标分析，失败样例 JSON 适合定位问题，HTML 页面适合演示。通过这种方式，Benchmark 不只是一次测试脚本，而是形成了可以复盘、可以对比、可以持续改进的评测体系。

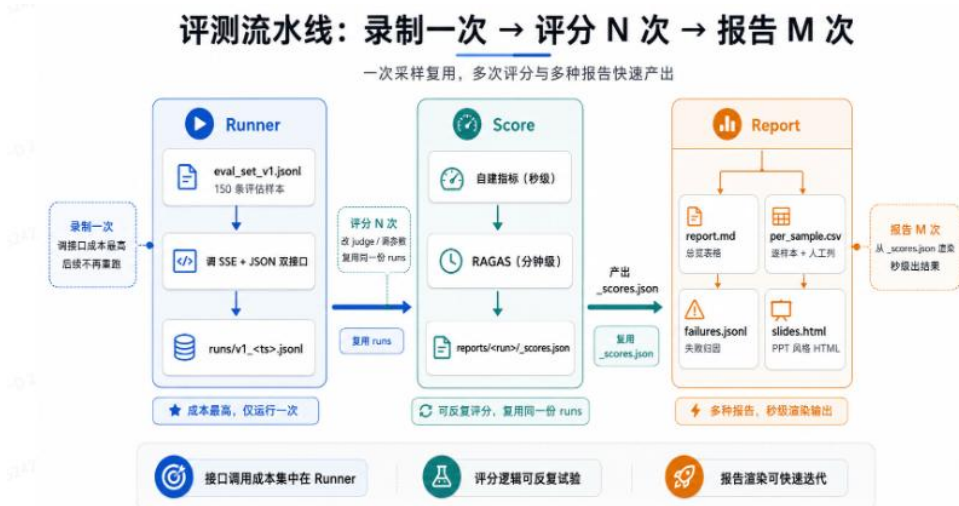


图 17: Benchmark 批量评测流程

5.5 当前测试结论

从当前测试结果看，KnowledgeOps Agent 已经覆盖 Agentic AI 原生开发方向中“企业知识库 + Agentic RAG”的核心要求。系统能够完成知识库管理、数据通道处理、意图管理、RAG 问答、链路追踪和 Agent 运营任务；能够在有知识库证据时生成带依据的回答，也能够没有相关文档时进入受控兜底；能够记录一次问答的 Trace，也能够记录一次 Agent 任务的 Run 和 Step。

评测体系方面，项目已经具备从单条问题到批量 Benchmark 的评估路径。单条问题评测能够记录 SSE 流式回答、首 token 时间、完整响应、命中文档和命中 Chunk；批量 Benchmark 能够通过问题集生成可复现的评测结果，并进一步输出报告、表格和失败样例。这说明系统不仅能演示功能，也具备一定的效果评估和问题定位能力。

后续需要继续完善的是评测数据规模和指标可信度。当前评测流程已经具备框架，下一步应准备更稳定的黄金问题集，补充人工标注的标准答案和相关证据，扩大不同知识库、不同问题类型、不同意图路径下的测试覆盖范围。同时，可以对比不同检索配置、不同向量后端、不同 Rerank 模型下的指标变化，使 Benchmark 结果更能体现系统优化效果。

六、系统升级与扩展

6.1 可扩展架构

KnowledgeOps Agent 当前已经形成了较完整的混合架构基础。系统并不是单一的 RAG 问答应用，而是由前端交互层、后端业务服务层、RAG 检索生成链路、Agent 自治运营链路、AI 基础设施层和数据基础设施层共同组成。前端负责知识库管理、问答、Trace、数据通道和 Agent 运营页面展示。后端负责文档处理、检索编排、模型调用、任务执行和运行记录。RAG 模块负责问题理解、混合检索、重排和回答生成。Agent 模块负责根据运营目标调用工具链，对知识库质量进行评估和诊断。

系统的扩展性首先体现在检索架构上。当前项目已经采用多通道混合检索思路，不是简单地只依赖向量相似度召回。用户问题进入系统后，会经过问题改写、会话记忆、意图识别，再进入全局向量检索和意图定向检索等通道。不同通道返回的候选结果会经过合并、去重和 Rerank 重排，最终再交给大模型生成回答。这样的设计更适合企业知识库场景，因为企业问题往往既需要语义匹配，也需要结合业务意图、知识库范围和上下文约束。后续扩展时，可以继续现有混合检索基础上优化通道权重、召回策略、结构化过滤、跨知识库路由和用户反馈学习，而不是重新设计检索链路。

系统的扩展性也体现在混合存储架构上。关系数据库用于保存知识库、文档、Chunk、会话、Trace、Agent Run、Agent Step 等结构化数据，保证业务记录可以查询、统计和复盘。向量索引用于保存知识片段的语义向量，支撑 RAG 问答中的相似度召回。项目同时保留 pgvector 和 Milvus 两类向量检索后端的适配能力，可以根据部署规模选择不同方案。对于课程演示和轻量部署，pgvector 能降低环境复杂度；对于更大规模的企业知识库，Milvus 可以承担更专业的向量检索任务。

Agent 层的扩展方式也比较清晰。系统已经采用 Planner、Executor、Tool Registry 和 Reporter 的分层设计。Planner 根据任务目标选择 workflow，Executor 按步骤执行工具，Tool Registry 统一管理工具实现，Reporter 将工具结果汇总为结构化报告。当前工具层已经覆盖知识库画像、文档新鲜度检查、知识检索、问题集 Benchmark、知识缺口分析、Chunk 质量检查、覆盖率评估和敏感信息检测等能力。后续如果要增加事实一致性检查、引用完整性检查、重复文档检测、主题聚类或自动知识补全，只需要新增 AgentTool 实现，并在工作流配置中编排使用，不需要改动 Agent 的核心执行框架。

6.2 下一阶段计划

首先是完善评估体系。当前系统已经具备 BenchmarkTool 和知识缺口分析工具，后续可以在此基础上进一步引入黄金问答集、人工相关性标注、召回率、命中率、平均 Top Score、低命中问题占比、事实一致性和引用完整性等指标。这样可以把系统评估从“能不能回答”提升到“回答是否有证据、证据是否相关、知识库是否覆盖业务问题”。对于企业知识库来说，这类指标比单次回答效果更重要，因为它能够指导后续文档补充和知识治理。

然后是增强工程可运行性。项目需要继续完善 README、环境变量说明、Docker Compose 启动步骤和常见问题处理方式，确保评阅者能够按步骤启动前后端和依赖服务。在工程配置上，可以继续补充生产 profile、密钥管理、模型限流、健康检查、日志规范和 CI/CD 流程。这样做的目的不是单纯让项目“看起来完整”，而是让系统从课程 Demo 更接近一个可部署、可维护、可排查的企业级应用。

加强前端运营体验。当前系统已经有 Agent Run 和 Step 的基础展示能力，后续可以增加报告导出、历史报告对比、弱问题看板、知识缺口工单流转和运营趋势分析。这样，Agent 评估结果就不只是一次性的报告，而能真正进入知识库维护流程。例如，系统发现某类问题长期低命中后，可以在前端形成待补充主题。管理员补充文档后，再通过 Benchmark 验证改进效果，从而形成持续迭代闭环。

6.3 AI 能力演进路径

短期内，系统的重点应放在稳定性和可解释性上。也就是说，先保证文档入库稳定、检索结果可追踪、兜底回答边界清晰、Agent 工具链可复现、Run 和 Step 记录完整。这个阶段不应盲目追求复杂的自动规划，而应确保每一次问答和每一次 Agent 任务都能说明输入是什么、执行了哪些步骤、得到了什么结果、失败时原因在哪里。

中期可以提升 Planner 和 Reporter 的智能化程度。Planner 可以从当前较明确的工作流选择，逐步升级为 LLM Planner 与规则校验结合的方式。LLM 负责理解管理员任务目标，生成候选工具链；规则层负责检查工具调用顺序、参数合法性和最大步骤数，避免 Agent 任意执行不可控操作。Reporter 也可以从结构化字段汇总，升级为面向管理者的自然语言报告，把 Benchmark 指标、低命中问题、知识缺口和优化建议解释得更清楚。

长期来看，KnowledgeOps Agent 可以演进为具备长期记忆和持续运营能力的企业知识库助手。系统可以持续跟踪新增文档、用户高频问题、失败问答、低置信检索和业务热

点变化，定期生成知识库健康报告，并主动提醒管理员哪些主题需要补充、哪些文档已经过期、哪些 Chunk 影响检索质量。到这个阶段，系统的价值就不只是回答问题，而是持续参与企业知识资产建设，使知识库从静态资料库变成可诊断、可评估、可优化的知识运营平台。

七、课程总结

7.1 个人收获

通过本项目，我对 Agentic AI 原生开发有了更具体的理解。最开始理解 Agent 时，我更容易把它看成“大模型调用工具”的过程，认为只要模型能够选择工具、调用接口、返回结果，就可以称为 Agent。但在真正实现 KnowledgeOps Agent 的过程中，我逐渐意识到，企业级 Agent 的核心并不只是“能调用工具”，而是要围绕任务目标、执行状态、工具边界、过程记录和结果评估设计一个完整系统。一个可用的 Agent 不应该只给出最终回答，还应该能够说明任务从哪里开始、经过了哪些步骤、每一步调用了什么工具、输出了什么结果、失败时错误发生在哪里。

本项目也让我更深入理解了 RAG 系统的复杂性。检索增强生成并不是“向量数据库 + Prompt + 大模型”这么简单。文档能否被正确解析，Chunk 切分是否合理，Embedding 是否适合当前语料，检索通道是否覆盖不同问题类型，Rerank 是否能筛选出真正相关的证据，Prompt 是否能约束模型基于知识库回答，低置信场景是否有兜底机制，这些都会影响最终效果。尤其是在企业知识库场景中，用户关心的不只是回答是否流畅，更关心回答是否有依据、依据是否来自企业文档、系统无法回答时能否明确说明原因。

在项目实现过程中，我对“知识库运营”这个概念也有了新的认识。传统知识库往往停留在文档上传和搜索阶段，维护质量主要依赖人工经验。但通过 BenchmarkTool、知识缺口分析、覆盖率评估、Run 和 Step 记录，我认识到知识库本身也可以被评估和治理。系统可以通过标准问题集发现哪些问题无法命中有效证据，通过低命中问题定位知识缺口，再反向指导文档补充、Chunk 调整和检索策略优化。这让我认识到，AI 项目的价值不应只体现在一次问答效果上，也应该体现在能否形成长期改进机制上。

7.2 工程思维转变

本项目让我明显感受到，从普通应用开发到 AI Agent 系统开发，工程思维需要发生变化。普通系统更强调业务流程、接口实现和数据增删改查，而 AI Agent 系统还必须考虑模型不确定性、检索质量、工具执行过程、失败回放和效果评估。如果只关注模型回答是否看起来正确，很容易忽略背后的工程风险。真正可维护的 AI 系统，需要把模型行为纳入可观测、可配置、可评估的工程框架中。

在这个过程中，我更加重视规格驱动开发。先明确 Product Spec、Architecture Spec、API Spec 和 Data Spec，再映射到模块划分、接口设计、数据对象、Agent 工具

和测试指标，可以减少后期开发中的随意性。比如 Agent 运行被抽象为 Run，工具执行被抽象为 Step，检索过程被抽象为 Trace，这些设计让系统不是只完成一次功能调用，而是能够记录完整的运行生命周期。这样的规格设计也让报告、测试和代码之间更容易对应起来。

另一个重要变化是从“模型中心”转向“系统中心”。大模型当然是系统能力的重要组成部分，但企业级应用不能把稳定性完全寄托在模型上。模型负责理解和生成，检索负责提供证据，工具负责执行可控动作，配置负责约束行为，Trace 和 Step 负责复盘，Benchmark 和知识缺口分析负责评估效果。只有这些部分共同工作，系统才具有可解释性和持续演进能力。通过这个项目，我更加理解了 AI 应用开发不是简单接入模型接口，而是把模型能力放进一个完整、可靠、可维护的工程系统中。

7.3 对课程的建议

我认为课程可以继续强化 SDD 方法论与 AI Agent 工程实践的结合。对于 AI 项目来说，规格设计尤其重要，因为很多问题不是写代码时才出现，而是在需求边界、工具边界、数据流和评估指标没有定义清楚时就已经埋下了。课程如果能够结合具体案例，讲解 Product Spec 如何映射到页面功能，Architecture Spec 如何映射到模块结构，API Spec 如何映射到接口契约，Data Spec 如何映射到数据库对象，会更有助于学生把文档设计和代码实现联系起来。

另外，课程可以增加更多 Agent 工程化案例，例如 RAG 评估、工具调用安全、长期记忆、Trace 观测、Benchmark 构建、多 Agent 协作和模型路由等内容。这些内容和真实企业级 AI 应用关系很密切，如果课程中能提供一些可复用的 Skill、插件或工程模板，例如文档解析模板、Agent 工具模板、RAG 评估模板、报告生成模板，也会让我感觉更容易把精力放在系统设计和业务创新上。

最后，我建议课程可以鼓励在最终报告中不仅展示“做了什么功能”，也展示“为什么这样设计”和“如何评估效果”。AI Agent 项目的难点往往不在于某个页面或接口，而在于系统是否能解释自己的行为、是否能发现失败原因、是否能持续优化。围绕这些问题进行课程训练，会更符合企业级应用软件设计与开发的目标。